03 操作系统进程生命周期

本册主线：从一个文件，到一次运行，再到退出

程序通常是一组指令及相关数据，可以保存在磁盘文件里。进程是程序的一次运行实例，操作系统为它维护执行状态、内存映射、打开的资源和身份等信息。

同一个 Python 文件可以被运行两次，形成两个进程；同一个模型文件也可能被多个服务读取。因此“文件有几份”“运行了几个进程”“存在几个 Agent 实例”是三个不同问题。

电脑里谁负责什么

部件	可以怎样理解	与进程的关系
CPU	执行指令的计算部件	当前执行某个线程的指令
内存 RAM	存放运行中代码和数据的工作区	为进程映射所需内容
磁盘 / 文件系统	保存长期数据	保存程序、模型与日志
操作系统内核	管理资源与权限的核心程序	创建、调度和回收进程
应用程序	用户想完成的具体工作	通过系统接口请求资源

一个够用的类比

程序像菜谱，进程像照着菜谱做菜的一次活动，CPU 像厨师，内存像工作台。一个菜谱能被用多次；某次活动结束，菜谱文件仍然存在。

类比也有边界：进程由真实的内核结构和硬件状态支持，不是一个单独的文件夹。删掉程序文件不等于可靠结束已运行的进程，结束进程也不等于删除所有持久化数据。

本册约定

概念主要按 Linux / Unix 解释；macOS 也能学习本册的进程与 POSIX 基础，但 Linux 特定接口不能直接照搬。Windows 底层创建接口不同。配套 Python 使用跨平台接口，不要求先学 C。图中的 PID 数字仅作示意。

必学：进程是什么、如何创建和退出、有哪些状态、如何通信。cgroup、namespace 等机制放在选读层，先理解用途与边界。

3.1 内核、系统调用与权限

应用为什么不能随意控制机器

现代操作系统通常把普通应用放在受限制的执行环境中。应用要读取文件、启动进程、发送网络数据或申请内存，通常通过系统调用进入内核，由内核检查参数、身份和权限，再执行对应操作。

```text
应用程序：希望打开文件 / 创建任务 / 发网络数据
                              |
                              v
系统调用入口：切换到内核处理相应请求
                              |
                              v
内核：身份与权限检查、资源分配、调用驱动
                              |
                              v
硬件 / 文件系统 / 设备 / 网络协议栈
```

用户态与内核态可先理解为不同权限的执行状态。一次系统调用通常让当前任务进入内核处理，并不必然发生“从进程 A 切换到进程 B”。这是权限切换和任务切换的区别。

进程通常有哪些信息

- PID：进程编号。PPID：父进程编号。
- 身份与权限：例如所属用户、允许访问哪些资源。
- 地址空间：它可以访问的虚拟地址范围及映射。
- 打开的文件与连接：例如日志、管道、套接字。
- 执行状态：运行位置、寄存器、可运行或等待状态等。

PID 是管理编号，不是永久身份，进程退出后编号可能被再次分配。长期安全归属若只记录一个数字，需要考虑编号复用和状态变化的竞态。

连接到课题

如果应用能够绕过某个用户态检查直接调用系统接口，那个检查就不能独立承担强制保护。研究控制点时，应问“所有相关请求是否都必须经过这里”“恶意程序能否修改执行规则”。

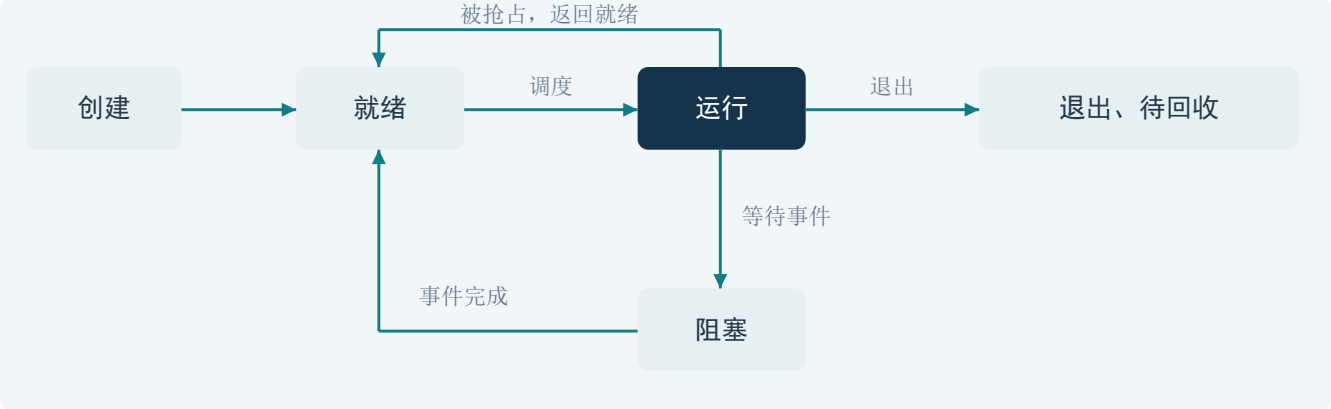
练习：某个 Agent 自己维护“是否允许启动工具”的变量，这个变量的可信程度取决于什么？

3.2 进程的状态与调度

有资格运行，不等于正在运行

状态	含义	生活类比
就绪	已具备运行条件，等待 CPU	排队等厨师
运行	此刻正在 CPU 上执行	厨师正在做这道菜
阻塞 / 等待	等待 I/O、时间或其他事件	等食材送到
已退出	执行已结束，可能仍待回收退出信息	工作结束，等登记结果

操作系统的调度器决定哪个可运行任务使用 CPU。时间片到期或更高优先级任务出现时，当前任务可能被切走；等待的事件完成后，任务通常先回到就绪，再等待调度。



上下文切换是保存当前任务的执行现场，再恢复另一个任务的现场。现场包括所需寄存器、下一条指令的位置及其他内核管理信息。一个核心上快速交替执行会呈现并发；多个核心还可能实现同时执行。

三个容易混淆的情况

进程调用 sleep 等待时间时，一般处于等待状态，而不是持续占满 CPU。进程“暂停”可以保留大量状态，之后可能恢复。进程“退出”则走资源清理与结果回收路径。

Linux 工具显示与教学状态图

ps 的 R 通常同时覆盖运行与可运行；S 是可中断睡眠；D 是不可中断睡眠，常与某些内核等待有关；Z 是僵尸状态。实际系统状态比入门四状态模型更细。

练习：某进程正在等待硬盘读完数据，它会一直占着 CPU 吗？事件完成后是否立刻在所有情况下运行？

3.3 创建、替换程序与等待

fork、exec、wait 各做一件不同的事

Linux / Unix 中，fork() 创建子进程。父子从 fork 后继续执行，但拿到不同返回值：父进程拿到子 PID，子进程拿到 0，失败时返回 -1。[01]

exec 系列接口把当前进程的程序映像替换成另一个程序。成功时通常不返回原程序，PID 保持不变。因此 exec 本身不能简单计为“新增了一个进程”。[02]

wait / waitpid 获取子进程状态；典型用法是父进程等待子进程退出，并让内核完成相应退出信息的回收。[03]

```text
父进程 P，运行程序 A
      |
      +--fork--> 子进程 C，起初也运行 A
      |                    |
      |                    +--exec--> C 改为运行程序 B
      |                                    |
      +--wait 等待-----------------------------+--exit
      |
      +--获得退出结果，继续运行
```

伪代码，只用于读流程

```text
child_pid = fork()
if 当前是子进程:
    exec(目标程序)
else:
    wait(child_pid)
```

父子谁先执行由调度决定，不应依赖“代码写在前面所以先输出”。实际程序还要处理失败和错误码。

Windows 与 Python 的对应关系

Windows 常用 CreateProcess 等接口创建程序实例，不能直接套用 fork 语义。Python 的 subprocess.Popen 把平台差异包装起来，适合先做跨平台实验；它在不同系统上的具体实现不保证都走同一条底层路径。[P1]

练习：一个 PID 从 Python 改为执行另一个可执行程序，PID 不变。这更接近 fork 还是 exec？

3.4 内存、虚拟地址与写时复制

两个进程里的“同一个地址”不一定是同一块内存

进程通常使用虚拟地址。CPU 的内存管理单元 MMU 根据操作系统配置的映射，把虚拟地址转换成物理内存位置，并检查相应访问权限。

```text
进程 A 的虚拟地址 0x1000 --> 映射 A --> 物理页 X
进程 B 的虚拟地址 0x1000 --> 映射 B --> 物理页 Y
```

因此不同进程可以使用相同的虚拟地址数字，却访问不同内容。页可以先理解为管理内存的一块固定大小单元，具体大小依平台和配置而异。

一个进程里常见的内容

代码区保存程序指令；栈保存函数调用中的部分临时数据；堆用于动态分配；映射区可能包含库、文件或共享内存。这里只建立位置概念，不要求记住地址布局。

fork 不一定立刻复制全部物理数据

Linux 采用写时复制 copy-on-write 优化普通 fork。父子在逻辑上拥有分开的地址空间，开始时一些物理页可共享；某一方写入需要私有的页时，再按规则复制相应页面。[01]

这解释了为何“创建了子进程”不能直接推断成“模型权重已物理复制了一整份”。大型模型还可能由共享推理服务提供，工具进程只向它发请求。

隔离也有例外与通道

共享内存允许进程在授权配置下访问共同区域；同一个文件、网络服务也能承载信息。拥有不同地址空间只是隔离的一部分，不能据此认定 Agent 无法协作。

选读：页表与保护

页表是描述映射和访问属性的数据结构。页表保护依赖内核正确配置与硬件执行；如果你的威胁模型允许内核本身恶意，就需要重新审视仅依赖内核的保护能否成立。

练习：两个进程都读取同一份只读模型文件，是否足以证明它们是同一个 Agent？说明理由。

3.5 进程、线程与 Agent 的关系

一个 Agent 可以跨越多个系统对象

线程是执行指令的单位；同一进程里的线程通常共享地址空间和许多资源，但各自保存线程执行现场。多个进程通常有分开的地址空间，也可能通过共享资源相互联系。

Agent 在本课题中可以先理解为：接收目标，借助模型选择行动，并通过工具持续执行的系统。harness 是组织提示、状态、工具、循环与错误处理的外围运行逻辑。

```TEXT
逻辑 Agent A
  |
  +--控制程序：保存任务、调用模型、决定下一步
  +--工具子进程：执行一次本地计算
  +--文件：任务状态、日志、可能的恢复材料
  +--网络连接：访问远端推理服务或外部工具

推理服务：可能同时服务 Agent A 与 Agent B
```

不能直接画等号的几组对象

- 一个模型权重文件不等于一个正在工作的 Agent。
- 一个进程可能只负责 Agent 的某项工具任务。
- 一个服务进程可能支持多个逻辑 Agent 会话。
- 一个 Agent 的状态可能保存在进程之外，供后续恢复。

怎样选一个工作定义

为实验定义“受控 Agent 域”：一个逻辑身份，加上被分配给它的运行任务、授权资源与允许的通信关系。后续再明确身份如何可信地绑定到进程、线程、设备队列与工具请求。

这是待设计的管理抽象。把几个 PID 写进列表，只是简单原型，还不能解决共享推理服务、并发创建、迁移和身份伪造。

练习：A 和 B 共用一个推理服务时，直接杀掉服务进程会有什么副作用？如果只按 PID 管理身份，会遗漏什么区分？

3.6 进程如何通信

地址空间分开，也可能交换信息

进程间通信简称 IPC。操作系统提供多种通信机制；应用还可通过共同文件或网络服务间接通信。对隔离课题，需要列出你准备控制的具体渠道。

渠道	信息怎样移动	本课题要检查什么
管道 pipe	一端写入字节，另一端读取	两端是否属于被允许通信的域
共享内存	共同访问一块内存	谁能建立映射、谁能读写
文件	一个写文件，另一个读取	目录权限、共享位置与继承句柄
本地套接字	同机端点交换消息	端点身份和访问权限
网络套接字	通过网络协议传输	目的地址、代理与外部中转
继承资源	子任务沿用已有资源引用	创建任务时继承了什么

文件描述符可以先理解为进程使用某个打开资源的编号；Windows 对应概念常称句柄。进程创建或程序替换时，某些资源可能被保留或继承，具体由接口与标志决定。

例：只拦网络还不够

假设 A 与 B 无法通过 TCP 发消息，但都能读写同一个目录。A 把文本写入文件，B 读取后就获得了信息。这是显式通道遗漏，无须先讨论复杂侧信道。

选读：隐蔽通道

即使显式 IPC 被控制，进程仍可能利用共享资源的时序变化传递少量信息。例如争用缓存影响另一方测到的时间。覆盖所有隐蔽通道通常是更强、更难的研究目标，应单独写入范围与假设。

练习：要验证“在已声明的显式渠道上隔离 A 与 B”，你会准备哪三类失败尝试？合法通信需要哪些对照？

3.7 退出、终止与回收

发出终止命令后，还需要观察结果

进程可以主动正常退出，也可以被外部机制终止。Linux 的 kill 系统调用用于发送信号，名称虽叫 kill，并非每种信号都表示终止。[04]

SIGTERM 常用于请求终止，程序可以处理、忽略或阻塞它；SIGKILL 不能被目标程序捕获、忽略或阻塞。但某些内核等待状态下，任务不保证在严格的短时间上界内立即消失。[04]

典型的父子回收路径

```text
子进程结束执行
      |
      v
内核保留少量退出信息：PID、状态等
      |
      v
父进程 wait / waitpid 获取结果
      |
      v
相应记录被回收
```

通常，已退出但还未被等待回收的子进程叫僵尸进程。它没有在继续执行用户代码，不能靠重复发终止信号让它“更死”；关键是回收路径。特定信号设置可使回收行为不同。[03]

父进程先退出会怎样

子进程不因父进程退出就普遍自动终止。Linux 中可能被其他回收者接管；Windows 也需要看显式设置的作业或管理机制。因此停止主程序不等于证明派生任务全部结束。

三个系统中的 Python 差异

Python terminate() 在 macOS / Ubuntu 等 POSIX 平台发送 SIGTERM，在 Windows 上调用 TerminateProcess；kill() 在 POSIX 上发送 SIGKILL，在 Windows 上是 terminate 的别名。不能把 Windows 的 terminate 理解成可处理的优雅退出请求。[P1]

练习：进程从工具列表消失、管道关闭、网络请求停止，这三种现象为什么不能互相完全替代？

3.8 实验一：创建一个子进程并等待

使用 Python，不需要先写 C

先运行配套 process_demo.py。下面是相同思路的简化版，便于逐行理解：

```python
import os
import subprocess
import sys

print("parent pid=", os.getpid(), flush=True)
code = "import os,time; print(os.getpid()); time.sleep(0.3)"
child = subprocess.Popen([sys.executable, "-u", "-c", code])
print("child pid=", child.pid, flush=True)
result = child.wait(timeout=5)
print("exit code=", result)
```

import 使用标准库；os.getpid() 获取当前 PID；sys.executable 是当前 Python 解释器路径。列表 [...] 把程序和每个参数分开传递。-c 后面是一小段要执行的 Python 程序，-u 让输出更及时。

Popen 启动子进程并返回管理对象。wait(timeout=5) 最多等待约 5 秒，否则报告超时；它本身不承诺超时就杀掉子进程。退出码 0 在本例表示子程序正常完成。

运行命令

```text
python3 process_demo.py
```

上面适用于 macOS 与 Ubuntu；Windows PowerShell 用 py -3 process_demo.py，已有 python 命令也可使用。预期输出含父子 PID、child done 及退出码 0。数字和部分输出顺序每次可能不同。

手工画图

把父 PID 写为 P、子 PID 写为 C，画出 P → C 的创建关系，再标记子进程打印、sleep 等待、退出、父进程 wait 返回四个事件。

练习：把子程序的等待改成 1 秒，父进程的等待是否仍会正常结束？timeout 是上限还是固定睡眠时长？

3.9 实验二：观察通信与终止

一条管道就能传递信息

运行 pipe_demo.py，预期输出为：

```
TEXT
parent received: hello from child
```

核心代码是：

```
PYTHON
result = subprocess.run(
    [sys.executable, "-c", "print('hello from child')"],
    capture_output=True, text=True, check=True, timeout=5,
)
print("parent received:", result.stdout.strip())
```

capture_output=True 收集标准输出与错误；text=True 按文本处理；check=True 在非零退出码时报告错误。父进程通过管道获得子进程的输出，这就是一次实际通信。

只终止自己创建的单个子进程

运行 terminate_demo.py。它启动一个打印 ready 后等待 10 秒的子程序；父进程读到 ready 后调用 terminate，随后 wait。POSIX 上等待超过 3 秒会再尝试 kill 并回收；Windows 下两种终止方法的底层语义不同，见 3.7。

输出类似 child ended; exit code = ...。具体数值与平台有关，不要求相同。这个实验只观察单个、已知的子进程，并没有实现进程树或 Agent 域的全量停止。

你应该记录什么

实验	观察内容	能说明什么
process_demo	父子 PID 与 wait 返回	创建、等待和退出结果
pipe_demo	父进程收到了子字符串	分开的进程可以通信
terminate_demo	已知子进程结束并被等待	终止请求与回收要配合

实验文件有完整清理逻辑。不要把示例改成无限创建任务；练习目的只是观察有限的生命周期事件。

3.10 选读：系统怎样组织隔离与管理

先知道机制的职责，不急着配置

机制	主要帮助解决什么	不能单独保证什么
用户与文件权限	限定身份能访问哪些对象	自动识别所有逻辑 Agent
namespace	为进程提供不同的资源视图	自动限制全部资源用量与通道
cgroup	分组管理任务与资源	单独提供所有信息流隔离
seccomp	限制可执行的系统调用	自动理解业务级合法行为
容器	组合多种系统机制包装应用	内核不可信时仍天然安全

namespace 可先理解为不同的“可见世界”，例如不同网络视图；cgroup 是管理分组；seccomp 是系统调用过滤。容器通常组合这些机制，并非另一台物理机器。

Linux cgroup v2 提供对组及其后代任务的管理接口，包括特定条件下的整组终止机制。它已考虑部分并发创建等问题，值得作为后续研究基线。[05]

对硬件研究的启发

先研究已有机制能在“内核可信、管理配置受保护”的条件下保证什么，再找出你的威胁模型增加了哪些要求。例如：身份标签需要不可伪造？控制状态需要独立于普通应用？哪些 CPU、内存或设备访问必须检查标签？

不能从“进程很多”直接推出“软件机制都做不到”。硬件辅助的研究价值，需要与已有软件机制在同样假设下比较，说明新增保护与代价。

只读观察命令，Ubuntu / WSL 可选

```bash
ps -o pid,ppid,stat,comm -p $$
cat /proc/self/cgroup
```

第一行查看当前 shell 的 PID、父 PID、状态与名字，$$ 是 shell 的自身编号；第二行查看执行 cat 的任务所属 cgroup。输出随系统配置变化，不要求与示意图相同。

本周无需修改系统 cgroup、权限或容器配置。能把“身份、分组、通信权限、停止范围”四个问题分开说明即可。

3.10 macOS 读者的对应关系

哪些能直接练，哪些先理解概念

内容	macOS 的学习方式	Linux / Ubuntu 的学习方式
创建、等待与退出	运行本套 Python 示例	运行同一套 Python 示例
fork / exec / wait 概念	阅读流程，理解 Unix 基础	阅读流程并结合 Linux 实现
terminate 与 kill	Python 采用 POSIX 信号语义	Python 采用 POSIX 信号语义
进程列表	ps 或“活动监视器”	ps、top 等工具
/proc、cgroup、seccomp	原生 Mac 不执行对应 Linux 命令	在支持的 Linux 配置中学习

macOS 的内核与 Linux 不同。某些进程状态字母、工具选项和底层实现细节也可能不同。因此本册的 Linux 实现细节不能全部当作 macOS 内核说明。

macOS：只读观察当前 shell

在 Mac 终端执行：

```bash
ps -o pid,ppid,state,comm -p $$
```

这可以看到当前 shell 的编号、父编号、状态和命令。再运行 Python 实验，把它打印的父子编号画成图。

Ubuntu / WSL：Linux 专项观察

```bash
ps -o pid,ppid,stat,comm -p $$
cat /proc/self/cgroup
```

第二行依赖 Linux 的 procfs 和 cgroup 接口。Mac 上出现“找不到文件”并不表示你没学会进程，而是所用系统没有相同接口。

本周无需先安装 Linux

用 macOS 原生环境完成三个 Python 实验、生命周期图和技术目标草案即可。之后若要实际配置 cgroup、namespace 或 seccomp，再选 Linux 虚拟机或导师提供的远程环境。

测试范围：当前制作环境已运行 Windows 与 Ubuntu，macOS 步骤依据官方文档核对，尚未在实体 Mac 实测。具体安装与入口见 05。

3.11 自测题

请用自己的话解释

1. 程序文件、模型权重、进程和 Agent 各是什么？能否一一对应？
2. 就绪与阻塞有什么区别？等待完成后为什么还可能要排队？
3. 系统调用与进程上下文切换是否总是一回事？
4. fork 新增了什么？exec 替换了什么？wait 获得并回收什么？
5. 写时复制为什么让“子进程数”和“模型物理副本数”不同？
6. 两个进程地址空间不同，为何仍能协作？举两个渠道。
7. SIGKILL 不能被应用忽略，是否就能保证任意时间上界内完成整个系统的停止？
8. 什么是僵尸进程？它还在运行用户代码吗？
9. 父进程退出后，所有子进程是否必然退出？
10. PID 为什么不适合作为永不变化的全局身份？

场景题：控制程序与共享推理服务

Agent A 的控制进程创建工具 T；A 与 B 共同调用推理服务 M；T 发起了一个网络请求 N。现在收到停止 A 的命令。

请画图并回答：哪些对象归 A 管？可以直接终止 M 吗？A 的控制进程退出后，你还要观察什么？如果 N 已经到达外部机器，本地停止能否撤销它已造成的影响？

场景题：限定你的隔离声明

你控制了共享文件、共享内存与本地套接字，但没有控制外部网络。请写一句准确的实验结论，再写一句过度夸大的结论，并说明两者差别。

完成标准

在纸上画完整生命周期；运行三个有限 Python 示例或推演输出；解释进程退出与 Agent 安全目标之间的差距。

3.12 参考答案与阅读来源

自测答案

1. 程序是指令与数据；权重是模型参数；进程是一次运行实例；Agent 是实现目标与行动循环的逻辑系统。它们没有普遍的一一对应关系。
2. 就绪任务等 CPU；阻塞任务等事件。事件完成后一般先成为可运行任务，再由调度器选择。
3. 不总是。当前任务可进入内核完成系统调用后继续运行，并未换成另一个任务。
4. fork 创建子进程；exec 替换当前程序映像；wait 获得子进程状态，并在典型退出路径中完成相关回收。
5. fork 可共享只读或写时复制物理页，且模型可能由共享服务提供。
6. 例如文件、管道、共享内存、网络连接。
7. 不能。还需考虑内核状态、并发任务、设备、已发请求与远端执行。
8. 已退出但保留少量信息等待回收的任务；不再执行用户代码。
9. 不必然；需明确整组生命周期管理机制。
10. PID 会复用，命名空间中的视图也可能不同。

场景 A：明确 A 的控制任务与 T 的归属；对共享 M 应按会话、请求或上下文控制，直接终止会影响 B；还要看 T、排队请求、连接与持久化恢复。已发生的外部影响通常不能靠本地停止自动撤销。

场景 B：可说“在所测配置下，A 与 B 无法通过列出的三个受控渠道传递测试消息”；不能说“它们绝对无法共谋”。后者超出了覆盖的渠道和行为证据。

阅读来源

[01] fork：https://man7.org/linux/man-pages/man2/fork.2.html

[02] execve：https://man7.org/linux/man-pages/man2/execve.2.html

[03] wait / waitpid：https://man7.org/linux/man-pages/man2/waitpid.2.html

[04] 信号：https://man7.org/linux/man-pages/man7/signal.7.html

[05] cgroup v2：https://docs.kernel.org/admin-guide/cgroup-v2.html

[06] namespace：https://man7.org/linux/man-pages/man7/namespaces.7.html

[07] seccomp：https://man7.org/linux/man-pages/man2/seccomp.2.html

[P1] Python subprocess：https://docs.python.org/3/library/subprocess.html

先阅读本册解释，需要进一步核实时再打开对应原文，不要求从头读完全部手册。